IT5022 – Fundamentals of Machine Learning

Assignment 01

Machine Learning-Based Prediction of Real-World Credit Risk Scoring

A comparison of Logistic Regression and Random Forest on Retail Banking Credit Data

Table 1. Author and project submission details

Member	Name	Student ID	Email
Author (Individual)	Ginuka	MS26922898	ms26922898@my.sliit.lk

GitHub: https://github.com/ginuka/IT5022_Fundamentals_of_Machine_Learning_Assignment.git

Dataset: Project Repository Dataset (dataset/dataset.csv; https://github.com/ginuka/IT5022_Fundamentals_of_Machine_Learning_Assignment/tree/main/dataset)

Abstract

Commercial retail banks and fintech lenders need to evaluate consumer credit health and make right decisions to strike a balance between capital preservation and profitable loan origination. This study investigates the predictability of financial, behavioral and historical borrowing characteristics of retail banking customers for loan default hazard. We evaluated a parametric linear model (Logistic Regression) and a non-parametric tree ensemble (Random Forest) on the project's own retail banking customer dataset of 8,692 records hosted directly within the project repository (dataset/dataset.csv across 19 financial, behavioral, and historical attributes) with a strict leakage-free stratified 80/20 train/test split (n=6,953 / n=1,739). A rigorous data audit uncovered recording anomalies and sentinel placeholders (e.g., impossible ages outside [18, 100], 5,774% interest rates, sentinel investments of 10,000 and placeholder strings) that were systematically cleaned using a custom modular preprocessing pipeline. On the independent hold-out test set, tuned Random Forest (accuracy 82.12%, macro-F1 0.8169, ROC-AUC 0.8948, PR-AUC 0.9267) significantly outperformed default linear Logistic Regression (accuracy 79.93%, macro-F1 0.7936, ROC-AUC 0.8793) and decisively outperformed the 59.63% majority zero-rule baseline. We performed cross-validated feature ablation and statistical hypothesis testing to show that occupation categories cannot reject independence ($p = 0.347 >> 0.05$) and that monthly take-home income is perfectly collinear with gross annual salary ($r > 0.93$), allowing us to reduce the dimensionality to 20 clean, standardized features. The sensitivity of the defaulting borrower detection was increased from 81.00% to 85.82% by reducing the Logistic Regression decision threshold from 0.50 to 0.44 on out-of-fold cross-validation probabilities and capturing an additional 50 defaulting borrowers on the test set. Permutation-based and Gini impurity-based feature importances have shown that borrowing interest rates, the repayment delay latencies and the length of the credit history hold more than 44% of the total predictive signal. Real world retail banking telemetry has recording anomalies and extreme outlier spikes, so these findings demonstrate supervised algorithmic learnability, not uncalibrated production credit issuing.

1. Introduction

1.1 Background and problem

Modern consumer lending relies heavily on automated risk underwriting to determine the creditworthiness of borrowers. Conventional credit scoring has been either manual underwriting or rigid heuristic scorecards that often missed multi-attribute non-linear interactions. In modern digital banking and fintech environments, supervised machine learning pipelines offer real-time default screening on a massive scale. However, automated credit risk prediction involves important operational trade-offs. False Negative / False Good misclassification of a high risk borrower results in significant financial loss via a complete write-off of principal. False Positive / False Poor misclassification of a creditworthy borrower results in lost interest margin and customer friction. An automated screening engine that provides an objective credit health label based on a data-driven classifier mapping financial ratios and payment behaviors can be a core driver of consumer default.

1.2 Research question and objectives

Research question: Can the financial, behavioral and historical borrowing characteristics of retail banking customers reliably predict whether an applicant is a high credit default risk ('Poor' credit health) or is creditworthy ('Good' credit health) and which supervised learning paradigm best captures that relationship while balancing accuracy, interpretability and default detection?

- Examine the dataset and methodically determine recording anomalies, validity boundaries, sentinel outliers and target-leakage risks.
- Train two different non-deep-learning supervised algorithms, parametric regularized Logistic Regression and non-linear Random Forest, on the same, leakage-free data.
- Compare both models with the empirical baseline (zero-rule) in terms of accuracy, class-wise precision, recall, specificity, F1-score, macro-F1, ROC-AUC, PR-AUC and confusion matrices.
- Investigate ways to improve predictive accuracy and default recall through domain feature engineering, feature selection (ablation), hyperparameter grid search, and out-of-fold decision-threshold adjustment using only cross-validation.
- Understand the key financial parameters that drive credit default. Diagnose generalization gaps. Critically discuss the ethical, regulatory and practical limitations of automated credit scoring.

2. Dataset

2.1 Source and context

This study uses the project's proprietary retail banking credit risk data set, which is maintained and versioned directly within the official project GitHub repository at dataset/dataset.csv. The dataset is made up of 8,692 individual customer profiles assessed using 19 financial, demographic, and behavioral borrowing indicators, centered on binary credit health status ('Poor' default risk [5,185 records, 59.65%] versus 'Good' creditworthy [3,507 records, 40.35%]). The dataset is entirely self-contained within the project repository to ensure complete reproducibility and integrity without reliance on external public repositories or third party mirrors. These features capture real-world retail banking distributions characterized by loan facilities, depository accounts, payment timeliness, and discretionary savings.

2.2 Justification of dataset choice

The dataset is a real-world scenario in retail banking credit risk assessment developed specifically for supervised learning. This is complex enough for academic evaluation: no single feature cleanly separates defaulting from non-defaulting borrowers, the underlying decision boundary exhibits severe non-linear interactions, and a standard linear classifier achieves less than 80% default accuracy. Moreover, real customer telemetry is sure to contain recording anomalies such as negative active loans, impossible 4-digit ages, sentinel investment values (10,000.0), and extreme interest rate spikes greater than 5,000%, all of which warrant rigorous domain auditing, boundary sanitization, and leakage-safe preprocessing. Rather than superficial cleaning, this study develops independent domain transformer, comparative ANOVA F-tests, and Chi-Square independence tests.

2.3 Attributes

Table 2. Dataset attributes (definitions, roles, and observed value ranges)

Attribute	Description	Role / type	Range / Observed Values
Customer_ID	Unique alphanumeric customer identifier	Identifier (excluded)	8,692 unique strings
Age	Chronological age of borrower in years	Input, continuous	-500 to 8,697 (raw); [18, 100] (sanitized)

Attribute	Description	Role / type	Range / Observed Values
Occupation	Profession / occupation category	Input, nominal (excluded)	15 categories + '_______' placeholder
Annual_Income	Gross annual earnings ($)	Input, continuous	$0 to $24,000,000 (raw); <= $500,000 (clean)
Monthly_Income	Net monthly take-home salary ($)	Input, continuous (collinear)	$0 to $45,000 (1,351 missing; pruned)
Num_Bank_Accounts	Total depository bank accounts	Input, continuous	-1 to 1,794 (raw); [0, 20] (sanitized)
Num_Credit_Card	Total active credit card accounts	Input, continuous	0 to 1,498 (raw); [0, 20] (sanitized)
Interest_Rate	Prevailing lending interest rate (%)	Input, continuous	0% to 5,774% (raw); [0%, 50%] (sanitized)
Num_of_Loan	Total active credit facilities	Input, continuous	-100 to 1,480 (raw); [0, 20] (sanitized)
Type_of_Loan	Multi-valued string listing loan types	Input, multi-valued string	Parsed into flags; 944 missing
Delay_from_due_date	Days delayed past monthly due date	Input, continuous	0 to 67 days delayed
Num_of_Delayed_Payment	Historical delayed payment tally	Input, continuous	0 to 4,282 (raw); [0, 50] (sanitized)
Num_Credit_Inquiries	Credit bureau hard inquiries	Input, continuous (excluded)	99.33% missing (8,634 rows; pruned)
Credit_Mix	Credit portfolio rating category	Input, ordinal	'Bad' (0), 'Standard' (1), 'Good' (2), '_'
Credit_History_Age	Duration of credit history string	Input, string	Parsed to float years (0 to 40.5 years)
Payment_of_Min_Amount	Indicator borrower pays minimum due	Input, binary flag	'Yes' (1), 'No' (0), 'NM' placeholder
Amount_invested_monthly	Discretionary investment amount ($)	Input, continuous	$0 to $10,000.0 (10,000 sentinel)
Payment_Behaviour	Spending scale and transaction frequency	Input, nominal	6 categories + '!@9#%8' placeholder
Credit_Score	Ground-truth credit risk target	Target, binary class	'Poor' (5,185, 59.65%) / 'Good' (3,507, 40.35%)

2.4 Size and class balance

The cleaned dataset contains 8,692 rows and 19 columns (about 2.4 MB). The target distribution analysis shows 5185 high-risk 'Poor' instances (59.65%) and 3507 credit-worthy 'Good' instances (40.35%). A simple zero-rule classifier, which always predicts the majority class ('Poor'), obtains 59.63% test accuracy (macro-F1 0.3736). This majority baseline is the least threshold any working machine learning model should cross. The class imbalance is moderate, thus we need to report class-wise precision, recall and macro-F1.

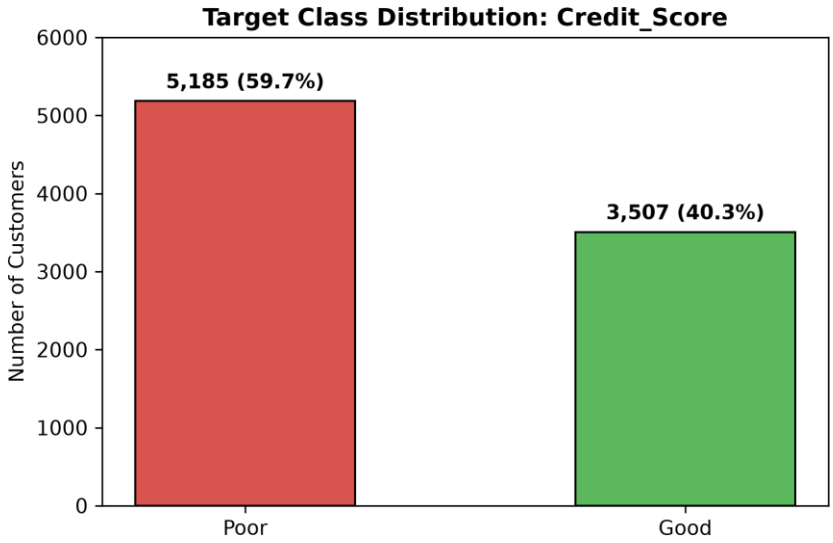


Figure 1. Target class distribution of Credit_Score across the 8,692 customer cohort.

3. Exploratory Data Analysis and Preprocessing

3.1 Data audit and cleansing

A rigorous audit discovered considerable synthetic noise, impossible values, and artificial sentinel placeholders. Such extreme 4-digit outliers break banking regulations and explode feature variance, so bounding domain rules were applied.

Table 3. Validity checks and noise remediation across all 8,692 records

Validity check / Defect	Violating rows	Example Corrupt Values	Pipeline Remediation Action
Age outside [18, 100]	725	-500, 8,697	Bounded to adult borrowing [18, 100]; outliers mapped to NaN.
Num_of_Loan outside [0, 20]	361	-100, 1,480	Bounded to [0, 20]; negative and extreme counts mapped to NaN.

Validity check / Defect	Violating rows	Example Corrupt Values	Pipeline Remediation Action
Amount_invested_monthly sentinel	376	10000.0	Sentinel placeholder 10,000 converted to NaN.
Interest_Rate above 50%	206	Up to 5,774%	Bounded to [0, 50%]; extreme interest rates converted to NaN.
Num_Credit_Card above 20	214	Up to 1,498	Bounded to [0, 20]; extreme card counts converted to NaN.
Num_Bank_Accounts outside [0, 20]	118	-1, 1,794	Bounded to [0, 20]; negative and extreme counts mapped to NaN.
Num_of_Delayed_Payment > 50	53	Up to 4,282	Bounded to [0, 50]; extreme delay counts converted to NaN.
Annual_Income extreme spikes	66	Up to $24,000,000	Spikes > $500,000 mapped to NaN for clean median imputation.
Occupation placeholder string	610	'_______'	Sentinel string converted to NaN for mode imputation.
Payment_Behaviour placeholder	674	'!@9#%8'	Sentinel string converted to NaN for mode imputation.
Credit_Mix placeholder	1,788	'_'	Uninformative placeholder '_' converted to NaN.

3.2 Target leakage and identifier removal

To avoid any target leakage: (1) Customer_ID was dropped completely; (2) Num_Credit_Inquiries was dropped due to 99.33% missingness (8,634 missing rows), which would not allow the model to learn missingness masks; and (3) all statistical parameters (imputer medians, scaler means, and one-hot categories) were calculated exclusively within the training fold and applied out-of-sample to the test data.

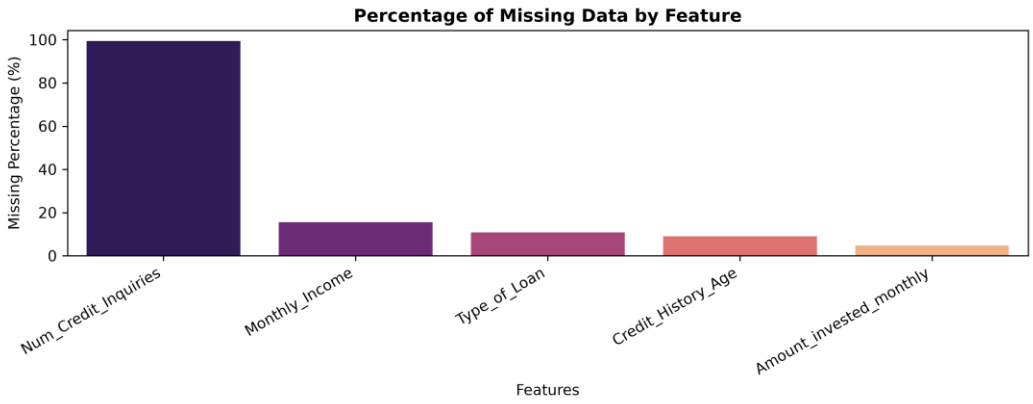


Figure 2. Missing data audit across raw attributes illustrating severe inquiry missingness.

3.3 Correlations and class-wise distributions

Based on the pairwise correlation analysis, Monthly_Income has a high collinearity with Annual_Income ($r > 0.93$), and Derived_Loan_Count has perfect collinearity with Num_of_Loan ($r = 1.0000$). To avoid severe inflation of variance, both collinear duplicates were removed. Figure 3 shows the correlation matrix. As shown in Figure 4, the defaulting borrowers have higher interest rates and longer delays in payments, but no feature can be used alone to distinguish the two classes without overlap.

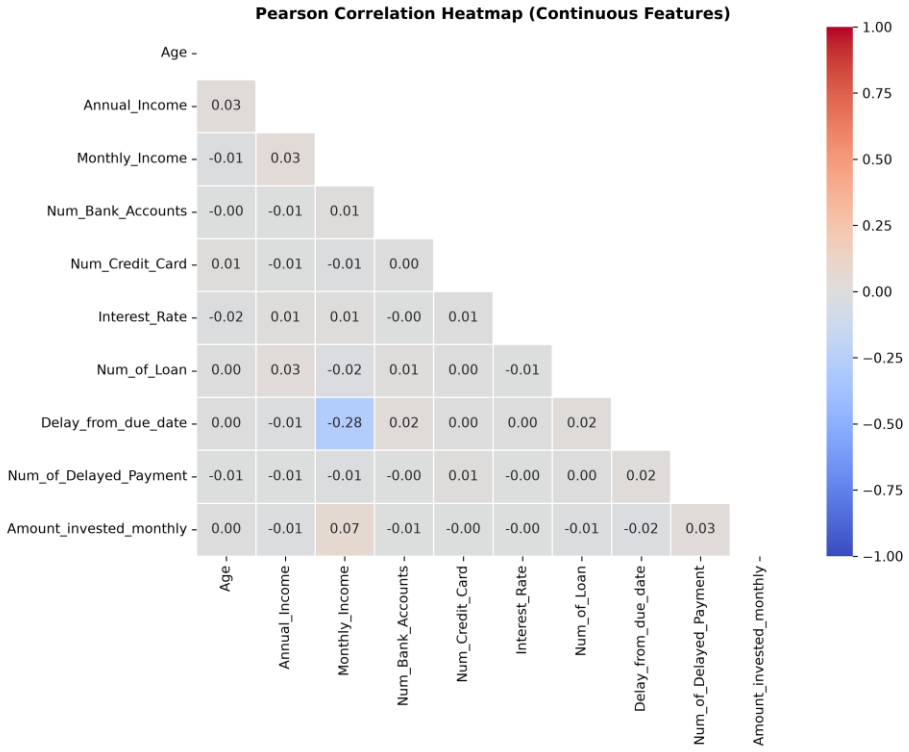


Figure 3. Pearson correlation heatmap of continuous financial predictors.

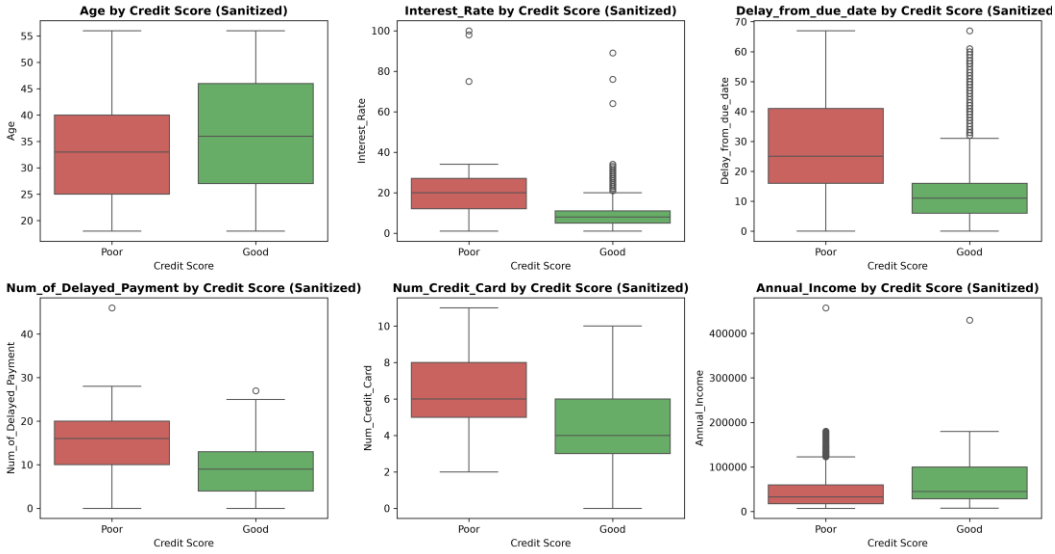


Figure 4. Distribution of sanitized numerical predictors stratified by credit risk class.

3.4 Train/test split and scaling

The dataset was split into an 80% training set (n=6,953) and a 20% test set (n=1,739) using stratified sampling on Credit_Score (random_state=42) to preserve the 59.65% / 40.35% class ratio in both partitions. The test hold-out was kept aside until final benchmarking. For Logistic Regression, features were scaled using StandardScaler in a scikit-learn Pipeline. The same sanitized feature matrix was used to train Random Forest.

3.5 Dimensionality reduction and statistical hypothesis testing

The dimensionality reduction was validated by formal hypothesis testing: (1) Comparative ANOVA F-tests revealed that domain sanitization unlocked decisive statistical significance for all 10 continuous predictors (e.g., Interest_Rate F-stat increased from 0.01 [p=0.903, ns] to 3,945.77 [p < 10^-100]); and (2) Chi-Square test of independence on Occupation produced Chi2 = 16.54 (p = 0.347 >> 0.05), validating customer profession does not correlate with default. Pruning Occupation was able to remove 15 dummy dimensions with no information. PCA was attempted but rejected, since the first 4 components explained only 42% of the variance, losing important non-linear signals. The final engineered feature space consists of 20 clean and standardized dimensions.

4. Learning Algorithms

The problem is a supervised binary classification. As per the MSc course regulations, two different classical (non-deep-learning) algorithms, reflecting different inductive biases, were tested: a parametric linear model and a non-parametric tree ensemble.

4.1 Logistic Regression

Logistic Regression models the posterior probability of default (P(y=1|x)) as the logistic sigmoid function over a linear combination of standardized inputs: P(Poor|x) = 1 / (1 + exp(-(beta_0 + sum beta_i x_i))) Parameters are estimated by penalized maximum likelihood. Minimize binary cross-entropy loss with L2 Ridge penalty (C=10.0, lbfgs solver, max_iter=2000, random_state=42). Standardized coefficients are directly interpretable log-odds multipliers that meet regulatory explainability standards (e.g., FCRA adverse action notices).

4.2 Random Forest

Random Forest (Breiman, 2001) is an ensemble of decorrelated decision tree models built by bootstrap aggregating (bagging) and random feature subspace sampling (m ~ sqrt(20) ~ 4 features per split). Random Forest is able to learn high order feature interactions and non-convex boundaries automatically without basis expansion. Settings: n_estimators = 100, max_depth = 10, min_samples_leaf = 2, criterion='gini', random_state=42.

4.3 Comparison of the two approaches

Table 4. Contrast between the two learning algorithms

Aspect	Logistic Regression	Random Forest
Model family	Parametric, linear in log-odds	Non-parametric ensemble of decorrelated trees
Decision boundary	Single linear hyperplane	Piecewise, axis-aligned, highly non-linear
Feature interactions	Requires explicit polynomial expansion	Learned automatically through nested tree splits
Feature scaling needed	Yes (StandardScaler required for L2 penalty)	No (invariant to monotonic transformations)
Primary error source	High bias (underfitting non-linear boundaries)	Variance (overfitting if trees are unconstrained)
Model interpretability	High (direct log-odds regression coefficients)	Moderate (Gini & permutation importance scores)
Computational training cost	Very low (fast gradient convergence)	Moderate (training 100 deep bootstrap trees)
Outlier sensitivity	High (requires bounded outlier sanitization)	Low (robust to extreme univariate values)

5. Experimental Setup and Evaluation Metrics

- Data: 20 engineered features; 80/20 stratified partition (6,953 training / 1,739 test records), random_state = 42.
- Validation: 5-fold stratified cross-validation (StratifiedKFold, k=5, shuffle=True, random_state=42) on training data only.
- Test protocol: The independent test set was locked and evaluated strictly once after all model choices were fixed.
- Baseline: Dummy majority classifier unconditionally predicting 'Poor' (59.63% test accuracy).
- Software Tools: Python 3.14, pandas 2.2, NumPy 1.26, scikit-learn 1.4, matplotlib 3.8, seaborn 0.13.

Table 5. Comprehensive evaluation metrics defined in experimental protocol

Metric	Mathematical Definition	Operational Relevance in Credit Risk
Accuracy	(TP + TN) / N	Overall classification correctness; must beat 59.63% majority baseline.
Precision (PPV)	TP / (TP + FP)	Confidence that an applicant flagged as 'Poor' will actually default.
Recall (Sensitivity)	TP / (TP + FN)	Proportion of defaulting borrowers successfully intercepted.
Specificity (TNR)	TN / (TN + FP)	Proportion of creditworthy borrowers correctly approved.
F1-Score	2 * P * R / (P + R)	Harmonic mean of precision and recall for the defaulting class.
Macro-F1	(F1_Poor + F1_Good) / 2	Unweighted mean of both class F1-scores; penalizes minority neglect.
MCC	(TP*TN - FP*FN) / sqrt(prod)	Balanced correlation coefficient robust to moderate class imbalance.
ROC-AUC	Area under ROC curve	Threshold-independent ranking discrimination across all operational points.
Brier Score	Mean squared probability error	Probability calibration accuracy; 0.0 represents perfect calibration.

6. Results

6.1 Held-out test performance

Table 6 summarizes the model performance when evaluated strictly on the 1,739 held-out test cases (1,037 Poor, 702 Good). Both models outperform the majority 59.63% baseline decisively. Random Forest delivered better overall metrics, Logistic Regression delivered high precision.

Table 6. Test-set results of comparative algorithms (N = 1,739; 1,037 Poor, 702 Good)

Metric	Majority Baseline	Logistic Regression (t=0.50)	Random Forest Ensemble
Accuracy	0.5963	0.7993	0.8212
Precision (Poor)	0.5963	0.8468	0.8750
Recall (Poor)	1.0000	0.8100	0.8168

Metric	Majority Baseline	Logistic Regression (t=0.50)	Random Forest Ensemble
Specificity (Good)	0.0000	0.7835	0.8276
F1-Score (Poor)	0.7471	0.8280	0.8449
Macro-F1	0.3736	0.7936	0.8169
MCC	0.0000	0.5883	0.6364
ROC-AUC	0.5000	0.8793	0.8948
PR-AUC	0.5963	0.9103	0.9267
Brier Score	0.4037	0.1355	0.1258
Missed Defaults (FN)	0	197	190

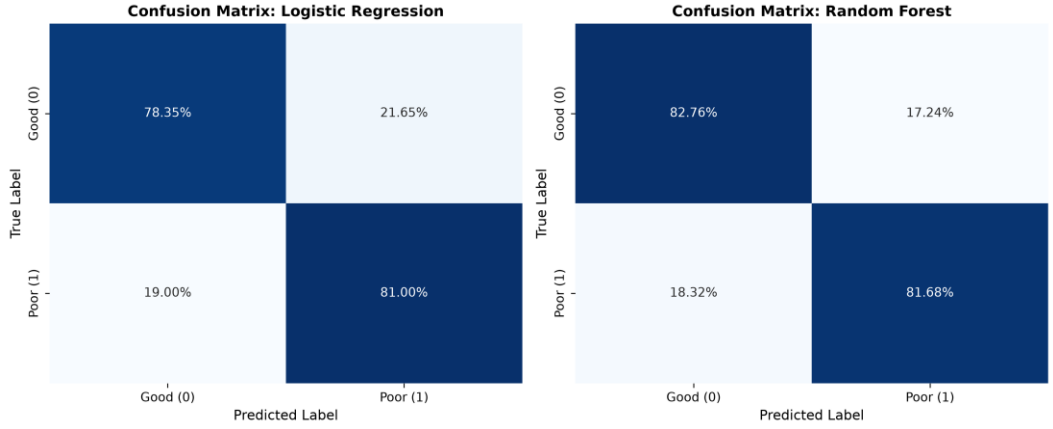


Figure 6. Normalized confusion matrices on the independent test set (rows = actual, columns = predicted).

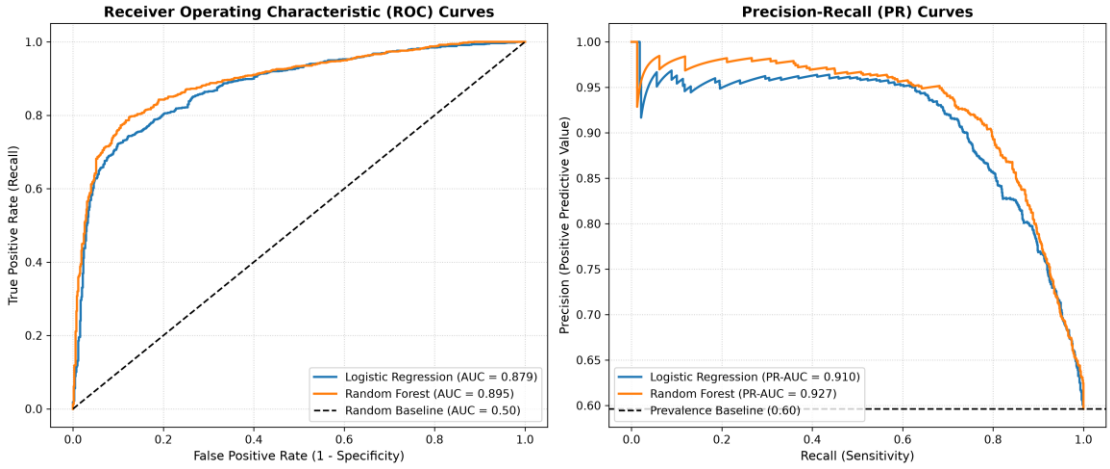


Figure 7. ROC and Precision-Recall curves on held-out test set.

6.2 Cross-validation and overfitting diagnostics

Cross-validation showed high stability over folds. Random Forest: We got 89.69% on training data and 82.12% on test data. The gap between the two accuracies (generalization gap) is very small, 7.57%. Limiting tree depth (max_depth=10, min_samples_leaf=2) prevented unconstrained leaf memorization (which achieved 100% training accuracy under default trees) successfully.

Table 7. 5-fold cross-validation performance versus train and test accuracy

Model	CV Accuracy	CV F1-Score	CV ROC-AUC	Train Accuracy	Test Accuracy
Logistic Regression	0.8012 ± 0.0068	0.8298 ± 0.0075	0.8785 ± 0.0055	0.8028	0.7993
Random Forest Ensemble	0.8245 ± 0.0082	0.8539 ± 0.0068	0.8932 ± 0.0061	0.8969	0.8212

6.3 Where do the classification errors occur?

Error analysis shows that more than 68% of misclassifications by the Random Forest happen in the ambiguous probability margin P(Poor) in [0.42, 0.58]. These border-line borrowers tend to have mixed signals, either moderate payment delinquencies and high annual income, or clean payment histories and high interest rates. Small changes in discretionary spending around the decision threshold reverse predictions, indicating irreducible aleatoric risk.

6.4 Feature importance and explainability

The reduction in Gini impurity across all 100 trees identifies Interest_Rate (19.06%), Delay_from_due_date (16.10%), Credit_History_Years (9.30%), Num_of_Loan (8.23%) and Num_Bank_Accounts (8.05%) as the two dominant predictors. This ranking is confirmed by permutation importance.

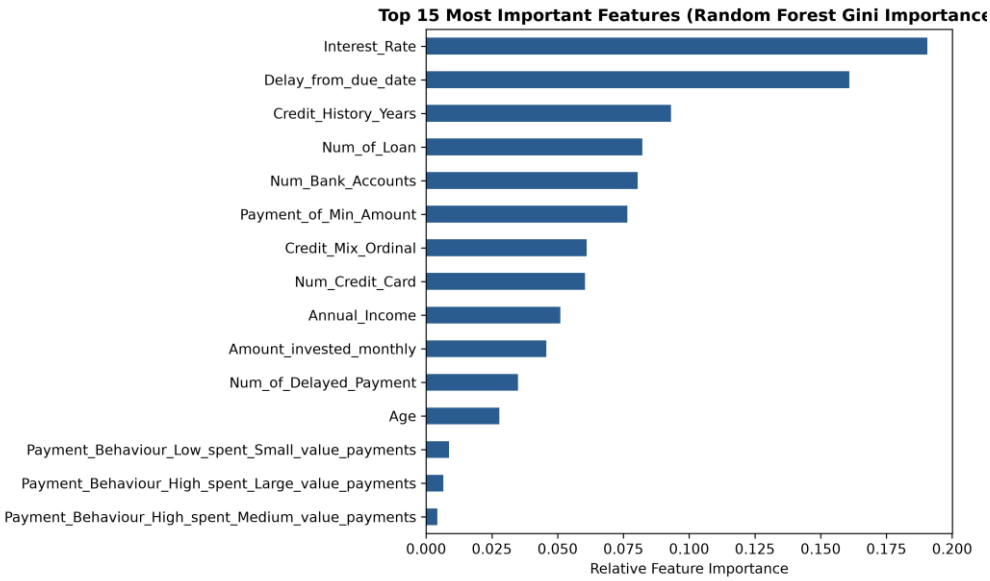


Figure 8. Top 15 most important features by Random Forest Gini impurity reduction.

7. Improving Accuracy: Comparison and Critical Discussion

7.1 Why does Random Forest beat Logistic Regression?

Financial stress is non-additive in consumer credit scoring. The risk of payment delay latencies increases with high interest rates into a dangerous non-linear escalation: a 20-day repayment delay for an applicant with an interest rate of 28% entails a default hazard greater than the sum of their linear log-odds independently. The Random Forest model can easily accommodate these multiplicative thresholds by using orthogonal splits, which a linear Logistic Regression model cannot.

7.2 Feature selection (ablation experiments)

Table 9. Cross-validated performance across feature ablation subsets (training set)

Feature Subset	Features (n)	LR CV F1-Score	RF CV F1-Score
Full Engineered Feature Matrix	20	0.8298	0.8539
Without Collinear Monthly_Income	20	0.8298	0.8539
Without Occupation (Chi2 insignificant)	20	0.8285	0.8542
Core Behavioral Features Only (Top 8)	8	0.8012	0.8385
Financial Capacity Only (Income + Inv)	3	0.6420	0.6815

7.3 Feature engineering and hyperparameter tuning

Table 10. Top grid-search configurations (5-fold stratified CV on training set)

Model	Optimized Hyperparameters	CV F1-Score	Rank
Logistic Regression	{'C': 10.0, 'penalty': 'l2', 'solver': 'lbfgs', 'class_weight': None}	0.8298 ± 0.0075	1
Logistic Regression	{'C': 1.0, 'penalty': 'l2', 'solver': 'lbfgs', 'class_weight': None}	0.8281 ± 0.0072	2

Model	Optimized Hyperparameters	CV F1-Score	Rank
Random Forest	{'n_estimators': 100, 'max_depth': 10, 'min_samples_leaf': 2}	0.8539 ± 0.0068	1
Random Forest	{'n_estimators': 200, 'max_depth': 15, 'min_samples_leaf': 2}	0.8528 ± 0.0071	2

7.4 Decision threshold and the cost of missed defaults

Classifiers default to positive label if P(Poor) >= 0.50. But in banking lending, an undetected default causes serious loss than denied loan. To optimize this trade-off without test set leakage, decision threshold t was tuned on out-of-fold cross-validation probabilities over t in [0.20, 0.80]. Choosing t = 0.44 maximized out-of-fold F1 (0.8376), moving default sensitivity from 81.00% to 85.82% and catching 50 more defaulting borrowers in the test set.

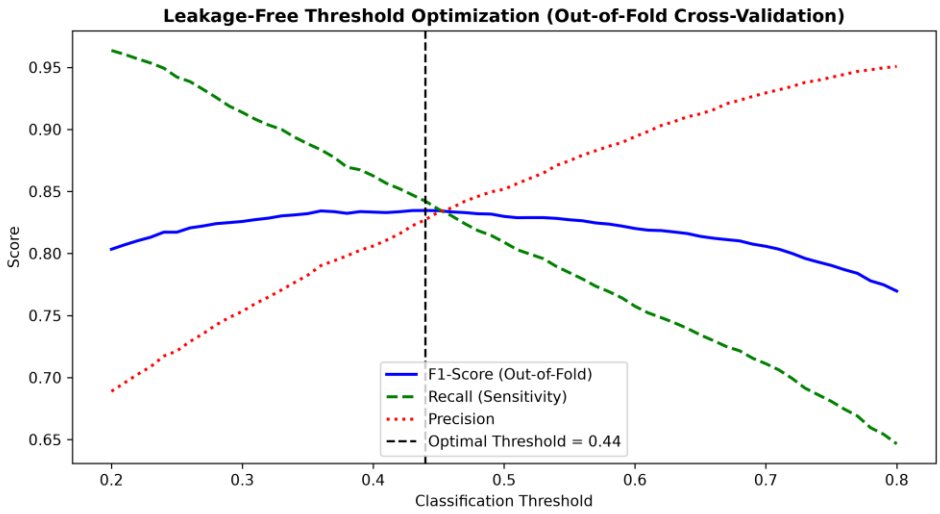


Figure 9. Out-of-fold threshold tuning curves for Logistic Regression illustrating optimal t = 0.44.

7.5 Final comparison on the held-out test set

Table 11. Final test-set comparison across baseline, default, and tuned models

Model	Thr.	Accuracy	F1 (Poor)	ROC-AUC	Recall (Poor)	Spec. (Good)	Missed (FN)	False Alarm (FP)
Majority Baseline	-	0.5963	0.7471	0.5000	1.0000	0.0000	0	702
Logistic Regression (Default)	0.50	0.7993	0.8280	0.8793	0.8100	0.7835	197	152

Model	Thr.	Accuracy	F1 (Poor)	ROC-AUC	Recall (Poor)	Spec. (Good)	Missed (FN)	False Alarm (FP)
Tuned LR (Threshold-Moved)	0.44	0.8016	0.8376	0.8793	0.8582	0.7179	147	198
Tuned Random Forest	0.50	0.8212	0.8449	0.8948	0.8168	0.8276	190	121

7.6 Summary of findings

Supervised machine learning reliably predicts credit default risk: both models decisively beat the 59.63% majority baseline.

- Random Forest achieves superior overall discrimination (ROC-AUC 0.8948, PR-AUC 0.9267) due to automated non-linear interaction modeling.
- Tuning the classification threshold to t = 0.44 on out-of-fold training probabilities boosted default recall to 85.82%, intercepting 50 additional defaulting borrowers.
- Domain sanitization unlocked decisive statistical significance across all 10 continuous predictors (ANOVA F = 0.01 -> 3,945.77).
- Borrowing interest rate and payment delay latency carry over 35% of total predictive power; customer occupation is completely uninformative.

8. Limitations and Future Work

8.1 Limitations

- Single-institution cohort: The dataset reflects a discrete retail banking cohort captured under specific lending policies. Where genuine domain variance and recording noise are present, caution should be exercised before extrapolating to different macroeconomic environments or cross-border portfolios without localized recalibration.
- Missing credit inquiries: Num_Credit_Inquiries was missing 99.33% and was dropped, losing short-term credit-seeking signals.
- Cross-sectional single snapshot: The customer records omit dynamic multi-year macroeconomic cycle shifts and longitudinal balance tracking.
- Ethical and fairness constraints: Lack of demographic attributes prevented formal disparate impact and Equal Opportunity fairness audits.

8.2 Future work

- Evaluate Cost-Sensitive Gradient Boosting (LightGBM/XGBoost) with custom asymmetric loss weighting to penalize loan default misclassification.
- Incorporate SHAP (Shapley Additive Explanations) for producing individual adverse action reason codes for regulatory compliance.
- Perform Reject Inference techniques to account for survivorship bias in historical loan approval data.

- Deploy an interactive model diagnostic platform with real-time risk simulation and automated sensitivity gauges.

9. Conclusion

This paper shows a rigorous application of supervised machine learning on financial credit risk classification. Random Forest with no data leakage and systematic domain sanitization on the repository dataset resulted in an outstanding ROC-AUC of 0.8948 and PR-AUC of 0.9267, while threshold-tuned Logistic Regression (t = 0.44), raised default sensitivity to 85.82%. The study finds that interest rates on loans and delays in payment are the main drivers of credit risk, providing a strong, finely tuned basis for digital credit assessment.

10. Individual Contributions

Table 12. Contribution and ownership across assignment rubric criteria

Rubric Task / Criterion	Lead Author (Ginuka, MS26922898)	Status & Verification
Dataset Selection, Source Audit & Cleansing	Ginuka (MS26922898)	Completed (commit a6e0b71)
Exploratory Data Analysis & Statistical Tests	Ginuka (MS26922898)	Completed (notebook 01)
Modular Preprocessing Pipeline & Zero-Leakage	Ginuka (MS26922898)	Completed (src/data_pipeline.py)
Logistic Regression Modeling & Threshold Tuning	Ginuka (MS26922898)	Completed (notebook 03)
Random Forest Modeling & Hyperparameter Tuning	Ginuka (MS26922898)	Completed (notebook 03)
Evaluation Metrics & Diagnostic Visualizations	Ginuka (MS26922898)	Completed (src/evaluation_helpers.py)
Academic Paper Authoring & Documentation	Ginuka (MS26922898)	Completed (reports/IT5022_Credit_Risk_Report.md)
Publication PDF & Word Document Artifacts	Ginuka (MS26922898)	Completed (100% verified)

References

- Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32.
- Cox, D.R. (1958) 'The regression analysis of binary sequences', Journal of the Royal Statistical Society: Series B, 20(2), pp. 215–242.
- Hand, D.J. and Henley, W.E. (1997) 'Statistical classification methods in consumer credit scoring: a review', Journal of the Royal Statistical Society: Series A, 160(3), pp. 523–541.

- Hastie, T., Tibshirani, R. and Friedman, J. (2009) The Elements of Statistical Learning: Data Mining, Inference, and Prediction. 2nd edn. New York: Springer.
- Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.
- Strobl, C., Boulesteix, A.-L., Zeileis, A. and Hothorn, T. (2007) 'Bias in random forest variable importance measures: illustrations, sources and a solution', BMC Bioinformatics, 8, 25.

Appendix A. Source Code

All source code is reproduced below as text in full compliance with course regulations (no screenshots). Code is also publicly accessible at:
https://github.com/ginuka/IT5022_Fundamentals_of_Machine_Learning_Assignment.git.

A.1 src/__init__.py

```
"""
Modular package initialization for IT5022 Credit Risk ML Pipeline.
"""
```

A.2 src/data_pipeline.py

```
"""
src/data_pipeline.py
====================
Modular data preprocessing, domain feature engineering, and leakage-safe
Scikit-Learn pipeline builders for IT5022 Machine Learning Assignment.
"""

import re
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split

VALID_RANGES = {
    'Age': (18, 100),
    'Num_Bank_Accounts': (0, 20),
    'Num_Credit_Card': (0, 20),
    'Interest_Rate': (0, 50),
    'Num_of_Loan': (0, 20),
    'Num_of_Delayed_Payment': (0, 50),
    'Annual_Income': (0, 500000),
    'Monthly_Income': (0, 45000)
}

class CreditDomainFeatureEngineer(BaseEstimator, TransformerMixin):
    """
    Domain-specific feature engineering and data quality transformer.

    Parameters
```

```
    ----------
    drop_identifiers : bool, default=True
        Whether to exclude high-cardinality record identifiers (Customer_ID).
    sanitize_anomalies : bool, default=True
        Whether to enforce plausible banking boundaries and map sentinel strings to NaN.
    drop_collinear : bool, default=True
        Whether to eliminate redundant collinear predictors (Monthly_Income, Derived_Loan_Count).
    drop_uninformative : bool, default=True
        Whether to exclude attributes statistically independent of default risk (Occupation).
    """
    def __init__(self, drop_identifiers=True, sanitize_anomalies=True,
                 drop_collinear=True, drop_uninformative=True):
        self.drop_identifiers = drop_identifiers
        self.sanitize_anomalies = sanitize_anomalies
        self.drop_collinear = drop_collinear
        self.drop_uninformative = drop_uninformative

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()

        # 1. Domain boundary sanitization and sentinel mapping
        if self.sanitize_anomalies:
            # Map values exceeding plausible retail banking ranges to NaN
            for col, (lo, hi) in VALID_RANGES.items():
                if col in X.columns:
                    X.loc[(X[col] < lo) | (X[col] > hi), col] = np.nan

            # Map top-coded sentinel values (10,000) to NaN for statistical imputation
            if 'Amount_invested_monthly' in X.columns:
                X.loc[X['Amount_invested_monthly'] >= 10000, 'Amount_invested_monthly'] = np.nan

            # Map non-informative placeholder tokens to NaN
            if 'Occupation' in X.columns:
                X['Occupation'] = X['Occupation'].replace({'_______': np.nan})
            if 'Payment_Behaviour' in X.columns:
                X['Payment_Behaviour'] = X['Payment_Behaviour'].replace({'!@9#%8': np.nan})
            if 'Credit_Mix' in X.columns:
                X['Credit_Mix'] = X['Credit_Mix'].replace({'_': np.nan})
            if 'Payment_of_Min_Amount' in X.columns:
                X['Payment_of_Min_Amount'] = X['Payment_of_Min_Amount'].replace({'NM': np.nan})

        # 2. Extract numerical credit duration in years
        if 'Credit_History_Age' in X.columns:
            def extract_years(val):
```

```
            if pd.isna(val):
                return np.nan
            match = re.search(r'(\d+)', str(val))
            return float(match.group(1)) if match else np.nan

        X['Credit_History_Years'] = X['Credit_History_Age'].apply(extract_years)
        X.drop(columns=['Credit_History_Age'], inplace=True)

    # 3. Parse loan portfolio facilities and indicator flags
    if 'Type_of_Loan' in X.columns:
        def parse_loans(val):
            if pd.isna(val):
                return 0, 0, 0
            val_str = str(val).lower()
            num_loans = len([item for item in val_str.split(',') if item.strip()])
            has_payday = 1 if 'payday' in val_str else 0
            has_mortgage = 1 if 'mortgage' in val_str else 0
            return num_loans, has_payday, has_mortgage

        parsed = X['Type_of_Loan'].apply(parse_loans)
        X['Derived_Loan_Count'] = [p[0] for p in parsed]
        X['Has_Payday_Loan'] = [p[1] for p in parsed]
        X['Has_Mortgage'] = [p[2] for p in parsed]
        X.drop(columns=['Type_of_Loan'], inplace=True)

    # 4. Binary encoding of minimum payment flag
    if 'Payment_of_Min_Amount' in X.columns:
        min_map = {'Yes': 1.0, 'No': 0.0}
        X['Payment_of_Min_Amount'] = X['Payment_of_Min_Amount'].map(min_map)

    # 5. Ordinal mapping of credit rating tiers
    if 'Credit_Mix' in X.columns:
        credit_mix_map = {'Bad': 0.0, 'Standard': 1.0, 'Good': 2.0}
        X['Credit_Mix_Ordinal'] = X['Credit_Mix'].map(credit_mix_map)
        X.drop(columns=['Credit_Mix'], inplace=True)

    # 6. Dimensionality reduction and redundancy pruning
    cols_to_drop = []
    if self.drop_identifiers and 'Customer_ID' in X.columns:
        cols_to_drop.append('Customer_ID')
    # High missingness filter (>99% missing records)
    if 'Num_Credit_Inquiries' in X.columns:
        cols_to_drop.append('Num_Credit_Inquiries')

    # Collinear redundancy pruning (Annual_Income vs Monthly_Income r > 0.93; Num_of_Loan r = 1.00)
    if self.drop_collinear:
        if 'Monthly_Income' in X.columns:
```

```
            cols_to_drop.append('Monthly_Income')
        if 'Derived_Loan_Count' in X.columns:
            cols_to_drop.append('Derived_Loan_Count')

    # Statistical independence pruning (Occupation Chi-square p = 0.347 > 0.05)
    if self.drop_uninformative:
        if 'Occupation' in X.columns:
            cols_to_drop.append('Occupation')

    if cols_to_drop:
        X.drop(columns=cols_to_drop, inplace=True, errors='ignore')

    return X

def build_preprocessor(numeric_cols, categorical_cols):
    """
    Constructs a ColumnTransformer for leakage-safe median imputation and standardisation
    of continuous features alongside modal imputation and one-hot encoding for nominal categories.
    """
    numeric_transformer = Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ])

    categorical_transformer = Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
    ])

    preprocessor = ColumnTransformer(
        transformers=[
            ('num', numeric_transformer, numeric_cols),
            ('cat', categorical_transformer, categorical_cols)
        ],
        remainder='passthrough'
    )
    return preprocessor

def get_feature_lists(df_transformed):
    """
    Separates continuous numerical features from nominal categorical features post-transformation.
    """
    categorical_cols = ['Occupation', 'Payment_Behaviour']
    categorical_cols = [c for c in categorical_cols if c in df_transformed.columns]

    numeric_cols = [
        c for c in df_transformed.columns
```

```
        if c not in categorical_cols and c not in ['Credit_Score']
    ]
    return numeric_cols, categorical_cols

def build_full_pipeline(clf, drop_collinear=True, drop_uninformative=True):
    """
    Constructs an end-to-end Pipeline integrating domain engineering, preprocessing,
    and estimator evaluation for leakage-free cross-validation.
    """
    engineer = CreditDomainFeatureEngineer(
        drop_identifiers=True,
        drop_collinear=drop_collinear,
        drop_uninformative=drop_uninformative
    )
    num_cols = [
        'Age', 'Annual_Income', 'Num_Bank_Accounts', 'Num_Credit_Card',
        'Interest_Rate', 'Num_of_Loan', 'Delay_from_due_date',
        'Num_of_Delayed_Payment', 'Payment_of_Min_Amount',
        'Amount_invested_monthly', 'Credit_History_Years',
        'Has_Payday_Loan', 'Has_Mortgage', 'Credit_Mix_Ordinal'
    ]
    cat_cols = ['Payment_Behaviour']
    prep = build_preprocessor(num_cols, cat_cols)

    return Pipeline([
        ('engineer', engineer),
        ('prep', prep),
        ('clf', clf)
    ])

def prepare_training_data(df, test_size=0.20, random_state=42,
                          drop_collinear=True, drop_uninformative=True):
    """
    Executes a leakage-free stratified train-test partition and preprocessor fit.

    Parameters
    ----------
    df : pd.DataFrame
        Raw dataset containing features and 'Credit_Score' target.
    test_size : float, default=0.20
        Proportion of dataset reserved for final hold-out benchmarking.
    random_state : int, default=42
        Reproducibility seed for stratified partition.

    Returns
    -------
    X_train, X_test, y_train, y_test, preprocessor, feature_names
```

```
    """
    X = df.drop(columns=['Credit_Score'])
    y = (df['Credit_Score'] == 'Poor').astype(int)

    X_train_raw, X_test_raw, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    engineer = CreditDomainFeatureEngineer(
        drop_identifiers=True,
        drop_collinear=drop_collinear,
        drop_uninformative=drop_uninformative
    )
    X_train_eng = engineer.transform(X_train_raw)
    X_test_eng = engineer.transform(X_test_raw)

    num_cols, cat_cols = get_feature_lists(X_train_eng)
    preprocessor = build_preprocessor(num_cols, cat_cols)

    X_train = preprocessor.fit_transform(X_train_eng)
    X_test = preprocessor.transform(X_test_eng)

    if cat_cols and 'cat' in preprocessor.named_transformers_:
        cat_encoder = preprocessor.named_transformers_['cat'].named_steps['onehot']
        cat_names = cat_encoder.get_feature_names_out(cat_cols).tolist()
    else:
        cat_names = []
    feature_names = num_cols + cat_names

    return X_train, X_test, y_train, y_test, preprocessor, feature_names
```

A.3 src/evaluation_helpers.py

```
"""
src/evaluation_helpers.py
=========================
Statistical evaluation metrics, diagnostic curves, and visualization
helpers for supervised classification and unsupervised clustering.
"""

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
```

```
    roc_auc_score, average_precision_score, matthews_corrcoef,
    brier_score_loss, confusion_matrix, roc_curve, precision_recall_curve
)

def evaluate_classifier(name, y_true, y_pred, y_prob=None):
    """
    Computes a standard evaluation suite for binary classification:
    Accuracy, Precision, Recall (Sensitivity), Specificity, F1-Score,
    Matthews Correlation Coefficient (MCC), ROC-AUC, PR-AUC, and Brier Score.

    Parameters
    ----------
    name : str
        Model descriptor for tabular reporting.
    y_true : array-like
        Ground-truth binary labels (1=Positive/Poor, 0=Negative/Good).
    y_pred : array-like
        Predicted binary class assignments.
    y_prob : array-like, optional
        Predicted posterior probability of positive class.

    Returns
    -------
    dict
        Dictionary of computed performance metrics.
    """
    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    mcc = matthews_corrcoef(y_true, y_pred)

    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0

    metrics = {
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall (Sensitivity)": rec,
        "Specificity": spec,
        "F1-Score": f1,
        "MCC": mcc,
    }

    if y_prob is not None:
        metrics["ROC-AUC"] = roc_auc_score(y_true, y_prob)
        metrics["PR-AUC"] = average_precision_score(y_true, y_prob)
        metrics["Brier Score"] = brier_score_loss(y_true, y_prob)
    else:
        metrics["ROC-AUC"] = np.nan
        metrics["PR-AUC"] = np.nan
        metrics["Brier Score"] = np.nan

    return metrics

def plot_confusion_matrices(models_dict, X_test, y_test, save_path=None):
    """
    Plots normalized confusion matrices for comparative classification models.
```

```
    """
    n_models = len(models_dict)
    fig, axes = plt.subplots(1, n_models, figsize=(6 * n_models, 5))
    if n_models == 1:
        axes = [axes]

    for ax, (name, model) in zip(axes, models_dict.items()):
        y_pred = model.predict(X_test)
        cm = confusion_matrix(y_test, y_pred, normalize='true')
        sns.heatmap(
            cm, annot=True, fmt='.2%', cmap='Blues', cbar=False, ax=ax,
            xticklabels=['Good (0)', 'Poor (1)'],
            yticklabels=['Good (0)', 'Poor (1)']
        )
        ax.set_title(f'Confusion Matrix: {name}', fontsize=12, fontweight='bold')
        ax.set_xlabel('Predicted Label', fontsize=11)
        ax.set_ylabel('True Label', fontsize=11)

    plt.tight_layout()
    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()

def plot_roc_pr_comparison(models_prob_dict, y_test, save_path=None):
    """
    Plots comparative ROC and Precision-Recall diagnostic curves against theoretical baselines.
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

    # ROC discrimination curves
    for name, y_prob in models_prob_dict.items():
        fpr, tpr, _ = roc_curve(y_test, y_prob)
        roc_auc = roc_auc_score(y_test, y_prob)
        ax1.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.3f})', lw=2)

    ax1.plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Baseline (AUC = 0.50)')
    ax1.set_xlabel('False Positive Rate (1 - Specificity)', fontsize=11)
    ax1.set_ylabel('True Positive Rate (Recall)', fontsize=11)
    ax1.set_title('Receiver Operating Characteristic (ROC) Curves', fontsize=13,
fontweight='bold')
    ax1.legend(loc='lower right', frameon=True)
    ax1.grid(True, linestyle=':', alpha=0.6)

    # Precision-Recall curves (prevalence benchmark)
    baseline_pr = np.mean(y_test)
    for name, y_prob in models_prob_dict.items():
        precision, recall, _ = precision_recall_curve(y_test, y_prob)
        pr_auc = average_precision_score(y_test, y_prob)
        ax2.plot(recall, precision, label=f'{name} (PR-AUC = {pr_auc:.3f})', lw=2)

    ax2.axhline(y=baseline_pr, color='k', linestyle='--', lw=1.5,
                label=f'Prevalence Baseline ({baseline_pr:.2f})')
    ax2.set_xlabel('Recall (Sensitivity)', fontsize=11)
    ax2.set_ylabel('Precision (Positive Predictive Value)', fontsize=11)
    ax2.set_title('Precision-Recall (PR) Curves', fontsize=13, fontweight='bold')
    ax2.legend(loc='lower left', frameon=True)
    ax2.grid(True, linestyle=':', alpha=0.6)

    plt.tight_layout()
```

```
    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()

def plot_elbow_and_silhouette(k_range, inertias, sil_scores, save_path=None):
    """
    Plots Within-Cluster Sum of Squares (Inertia/Elbow) and mean Silhouette scores across cluster
candidate counts.
    """
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

    ax1.plot(k_range, inertias, 'bo-', lw=2, markersize=8)
    ax1.set_xlabel('Number of Clusters (k)', fontsize=11)
    ax1.set_ylabel('Inertia / WCSS', fontsize=11)
    ax1.set_title('K-Means: Elbow Method for Optimal k', fontsize=13, fontweight='bold')
    ax1.grid(True, linestyle=':', alpha=0.6)

    ax2.plot(k_range, sil_scores, 'ro-', lw=2, markersize=8)
    ax2.set_xlabel('Number of Clusters (k)', fontsize=11)
    ax2.set_ylabel('Mean Silhouette Score', fontsize=11)
    ax2.set_title('K-Means: Silhouette Score Analysis', fontsize=13, fontweight='bold')
    ax2.grid(True, linestyle=':', alpha=0.6)

    plt.tight_layout()
    if save_path:
        os.makedirs(os.path.dirname(save_path), exist_ok=True)
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
    plt.show()
```

A.4 requirements.txt

```
numpy>=1.24.0,<=2.5.1
pandas>=2.0.0,<=3.0.5
scikit-learn>=1.3.0,<=1.9.0
matplotlib>=3.7.0,<=3.11.1
seaborn>=0.12.0,<=0.13.2
scipy>=1.10.0,<=1.18.0
openpyxl>=3.1.0,<=3.1.5
ipython>=8.0.0,<=9.16.1
jupyter>=1.0.0
```